
INF1350

Sistemas Reativos

Modelos de Concorrência

Prof.: Adriano Branco

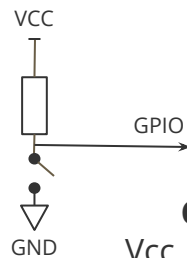
abranco@inf.puc-rio.br

Slides baseado em aulas anteriores da Prof. Noemi Rodriguez e com contribuição do Prof. Francisco Sant'Anna e do Prof. Adriano Branco

Exemplo - versão interativa

```
...  
// função loop executa para sempre  
// (chamada de loop do framework)  
void loop() {  
    digitalWrite(LED1, HIGH); // apagar o led (voltagem em HIGH)  
    delay(1000); // esperar um segundo  
    digitalWrite(LED1, LOW); // acender o led (voltagem em LOW)  
    delay(1000); // esperar um segundo  
    if (!digitalRead(KEY1)) {  
        digitalWrite(LED1, LOW);  
        exit(0);  
    }  
}
```

- pisca-pisca...
- parar quando usuário apertar a chave 1 e deixar a luz acesa para sempre?



Chave aberta
Vcc, +5v, 1, High, True

Chave fechada
GND, 0v, 0, Low, False

Programa interativo

Exemplo - versão interativa x reativa

```
int state = 1;
unsigned long old;
void setup () {
    ...
    old = millis();
    digitalWrite(LED1, state);
}
```

```
void loop () {
```

```
    unsigned long now = millis();
    if (now >= old+1000) {
        old = now;
        state = !state;
        digitalWrite(LED1, state);
    }
```

```
    int but = digitalRead(KEY1);
    if (!but) {
        digitalWrite(LED1, LOW);
        while(1);
    }
```



```
}
```

Exemplo - versão interativa x reativa

```
int state = 1;
unsigned long old;
void setup () {
    ...
    old = millis();
    digitalWrite(LED1, state);
}
```

```
void loop () {
```

```
    unsigned long now = millis();
    if (now >= old+1000) {
        old = now;
        state = !state;
        digitalWrite(LED1, state);
    }
```

```
    int but = digitalRead(KEY1);
    if (!but) {
        digitalWrite(LED1, LOW);
        while(1);
    }
```

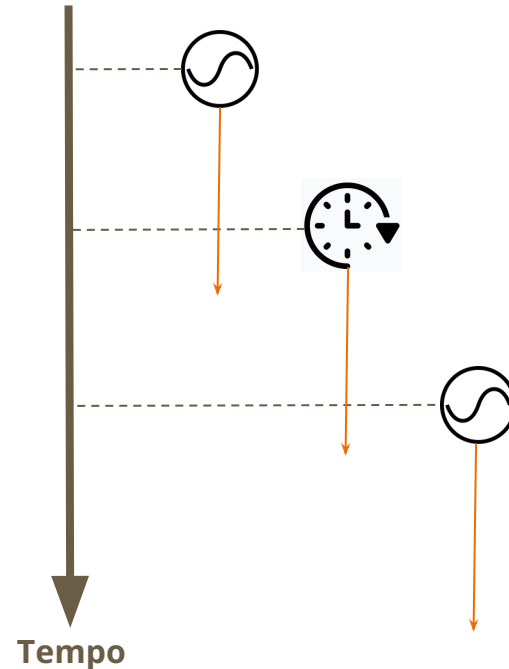
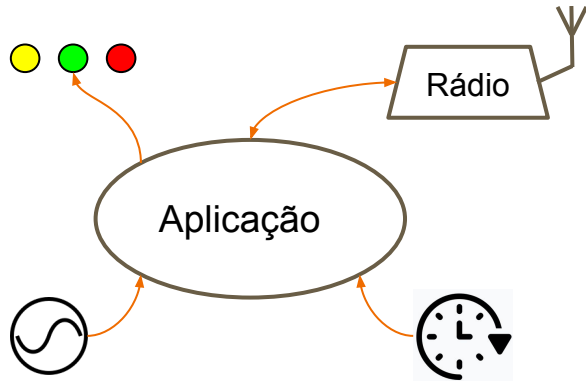
```
void loop () {
    unsigned long now = millis();
    if (now >= old+1000) {
        old = now;
        state = !state;
        digitalWrite(LED1, state);
    }
}
```

```
void loop () {
    int but = digitalRead(KEY1);
    if (!but) {
        digitalWrite(LED1, LOW);
        exit();
    }
}
```

```
}
```

Modelos de concorrência

- sistemas reativos devem estar prontos a reagir a diferentes “estímulos” do ambiente externo
- cada estímulo gera uma reação



Modelos de concorrência

```
int state = 1;
unsigned long old;
void setup () {
  ...
  old = millis();
  digitalWrite(LED1, state);
}
```

```
void loop () {
  unsigned long now = millis();
  if (now >= old+1000) {
    old = now;
    state = !state;
    digitalWrite(LED1, state);
  }
}
```

```
int but = digitalRead(KEY1);
if (!but) {
  digitalWrite(LED1, LOW);
  while(1);
}
```

Dificuldades quando reações diferentes acessam o mesmo recurso.

```
void loop () {
  unsigned long now = millis();
  if (now >= old+1000) {
    old = now;
    state = !state;
    digitalWrite(LED1, state);
  }
}
```

```
void loop () {
  int but = digitalRead(KEY1);
  if (!but) {
    digitalWrite(LED1, LOW);
    exit();
  }
}
```

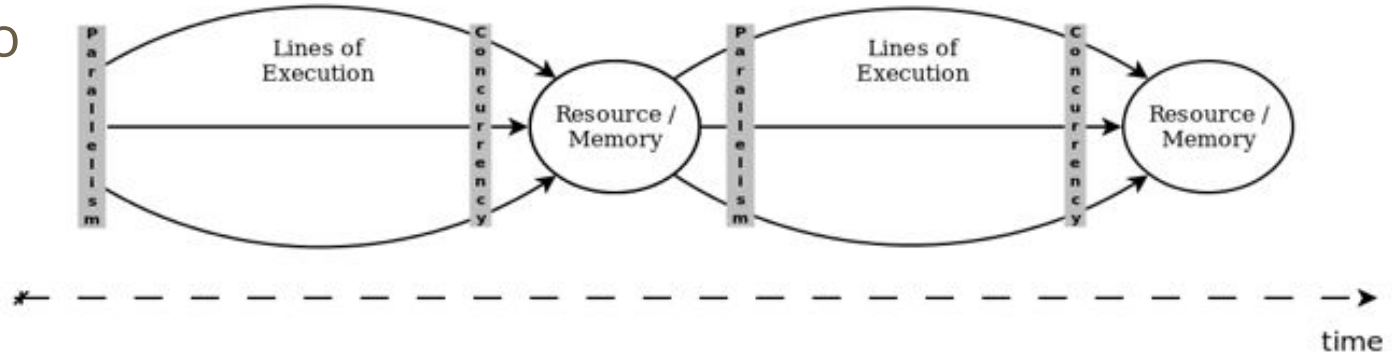
parênteses: concorrência e paralelismo

concorrência: atividades diferentes acessam recursos compartilhados

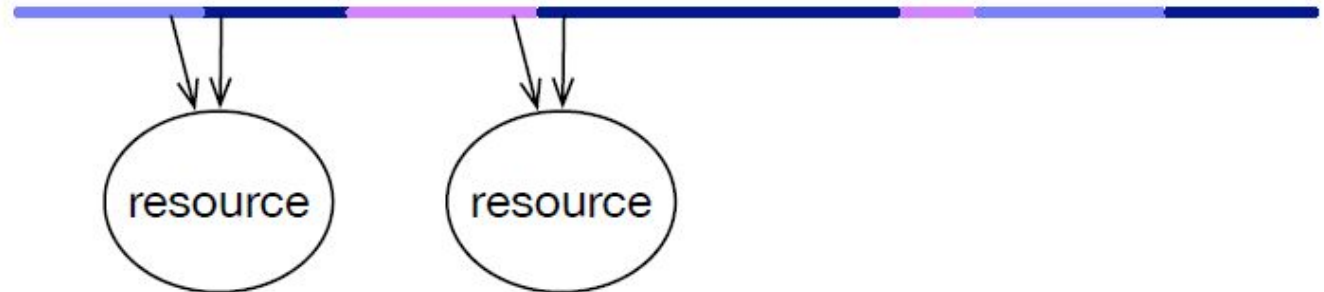
paralelismo: atividades diferentes em execução simultânea

parênteses: concorrência e paralelismo

concorrência no
paralelismo



concorrência
sem paralelismo



Concorrência e Paralelismo

- Rob Pike - “Concurrency Is Not Parallelism”
 - Concurrency is about dealing with lots of things at once.
 - Parallelism is about doing lots of things at once.
 - <https://www.youtube.com/watch?v=oV9rvDlKEg>

fecha parênteses ...

Modelos de Concorrência

- assíncrono
- síncrono

Modelos de Concorrência

- **assíncrono**

- execução independente e sincronização explícita
 - threads com mutexes ou semáforos
 - atores com troca de mensagens

- **síncrono**

- execução dependente e sincronização implícita
 - modelo setup e loop [?]
 - padrão observador
 - linguagens síncronas

um modelo assíncrono - exemplo chibiOS¹

Threads

- thread 1 lê chave
 - thread 2 controla timer de pisca-pisca
-
- threads - gasto de recursos escassos
 - sincronização e tempo de reação
 - escalonamento

[1] A versão mais atual do **chibiOS** evoluiu bastante e não é mais compatível com o Arduino Uno.

um modelo assíncrono - exemplo chibiOS

```
void Thread1 (void) {  
    <...>  
}
```

```
void Thread2 (void) {  
    <...>  
}
```

```
void setup() {  
    chThdCreateStatic(..., Thread1);  
    chThdCreateStatic(..., Thread2);  
}
```

um modelo assíncrono - exemplo chibiOS

```
int state = 1;
unsigned long old;
void setup () {
    old = millis();
    digitalWrite(LED1, state);
}
```

```
void loop () {
    unsigned long now = millis();
    if (now >= old+1000) {
        old = now;
        state = !state;
        digitalWrite(LED1, state);
    }
    int but = digitalRead(KEY1);
    if (!but) {
        digitalWrite(LED1, LOW);
        while(1);
    }
}
```

```
void Thread1 (void) {
    while (TRUE) {
        digitalWrite(LED1, HIGH);
        chThdSleepMilliseconds(1000);
        digitalWrite(LED1, LOW);
        chThdSleepMilliseconds(1000);
    }
}
```

```
void Thread2 (void) {
    while (TRUE) {
        int but = digitalRead(KEY1);
        if (!but) {
            digitalWrite(LED1, LOW);
            break;
        }
    }
}
```

```
void setup () {
    chThdCreateStatic(..., Thread1);
    chThdCreateStatic(..., Thread2);
}
```

um modelo assíncrono - exemplo chibiOS

- e como se relacionam os acessos ao LED?
- e o gasto de recursos...

```
void loop () {  
    unsigned long now = millis();  
    if (now >= old+1000) {  
        old = now;  
        state = !state;  
        digitalWrite(LED1, state);  
    }  
    int but = digitalRead(KEY1);  
    if (!but) {  
        digitalWrite(LED1, LOW);  
        while(1);  
    }  
}
```

```
void Thread1 (void) {  
    while (TRUE) {  
        digitalWrite(LED1, HIGH);  
        chThdSleepMilliseconds(1000);  
        digitalWrite(LED1, LOW);  
        chThdSleepMilliseconds(1000);  
    }  
}  
  
void Thread2 (void) {  
    while (TRUE) {  
        int but = digitalRead(KEY1);  
        if (!but) {  
            digitalWrite(LED1, LOW);  
            break;  
        }  
    }  
}  
  
void setup () {  
    chThdCreateStatic(..., Thread1);  
    chThdCreateStatic(..., Thread2);  
}
```

um modelo assíncrono - exemplo chibiOS

- e quando a thread 2 termina, o que ocorre com a tarefa 1?

```
void loop () {  
  unsigned long now = millis();  
  if (now >= old+1000) {  
    old = now;  
    state = !state;  
    digitalWrite(LED1, state);  
  }  
  int but = digitalRead(KEY1);  
  if (!but) {  
    digitalWrite(LED1, LOW);  
    while(1);  
  }  
}
```

```
void Thread1 (void) {  
  while (TRUE) {  
    digitalWrite(LED1, HIGH);  
    chThdSleepMilliseconds(1000);  
    digitalWrite(LED1, LOW);  
    chThdSleepMilliseconds(1000);  
  }  
}
```

```
void Thread2 (void) {  
  while (TRUE) {  
    int but = digitalRead(KEY1);  
    if (!but) {  
      digitalWrite(LED1, LOW);  
      break;  
    }  
  }  
}
```

```
void setup () {  
  chThdCreateStatic(..., Thread1);  
  chThdCreateStatic(..., Thread2);  
}
```

terminação de tarefas

- dificuldades...
- Java - “Why Are Thread.stop, Thread.suspend, Thread.resume, and Runtime.runFinalizersOnExit deprecated?”
- POSIX Threads - pthreads:
 - `man pthreads_cancel`
 - `man pthreads`
 - LinuxThreads → NPTL (Native POSIX Threads Library)

escalonamento de tarefas

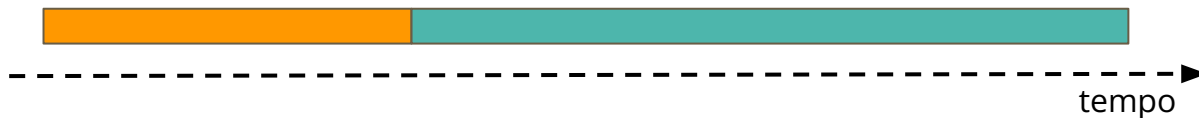
modelos



- preemptivo (assíncrono)



- não preemptivo (síncrono)



código com threads preemptivas e proteções

```
MUTEX_DECL(mut);
```

```
Thread* t1;
```

```
void Thread1 (void) {
```

```
    while (TRUE) {
```

```
        digitalWrite(LED1, LOW);
```

```
        chThdSleepMilliseconds(1000);
```

```
        if (chThdShouldTerminate())
```

```
            break;
```

```
        digitalWrite(LED1, HIGH);
```

```
        chThdSleepMilliseconds(1000);
```

```
        if (chThdShouldTerminate())
```

```
            break;
```

```
    }
```

```
}
```

```
void Thread2 (void) {
```

```
    while (TRUE) {
```

```
        int but = digitalRead(KEY1);
```

```
        if (!but) {
```

```
            digitalWrite(LED1, LOW);
```

```
            chThdTerminate(t1);
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
void setup () {
```

```
    t1 = chThdCreateStatic(..., Thread1);
```

```
    chThdCreateStatic(..., Thread2);
```

```
}
```

código com threads preemptivas e proteções

```
MUTEX_DECL(mut);
```

```
Thread* t1;
```

```
void Thread1 (void) {
```

```
    while (TRUE) {
```

```
        digitalWrite(LED1, LOW);
```

```
        chThdSleepMilliseconds(1000);
```

```
        chMtxLock(&mut);
```

```
        if (chThdShouldTerminate())
```

```
            break;
```

```
        digitalWrite(LED1, HIGH);
```

```
        chMtxUnlock(&mut);
```

```
        chThdSleepMilliseconds(1000);
```

```
        if (chThdShouldTerminate())
```

```
            break;
```

```
    }
```

```
}
```

```
void Thread2 (void) {
```

```
    while (TRUE) {
```

```
        int but = digitalRead(KEY1);
```

```
        if (!but) {
```

```
            chMtxLock(&mut);
```

```
            digitalWrite(LED1, LOW);
```

```
            chThdTerminate(t1);
```

```
            chMtxUnlock(&mut);
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
void setup () {
```

```
    t1 = chThdCreateStatic(..., Thread1);
```

```
    chThdCreateStatic(..., Thread2);
```

```
}
```

tarefas demoradas - como tratar?

```
void loop () {  
    digitalWrite(LED1, HIGH);  
    f();  
    digitalWrite(LED1, LOW);  
    f();  
}
```

```
void Thread1 (void) {  
    while (TRUE) {  
        f(); // operação longa  
    }  
}  
  
void Thread2 (void) {  
    while (TRUE) {  
        // pisca led!  
    }  
}
```

tarefas demoradas - como tratar?

```
void loop () {  
  digitalWrite(LED1, HIGH);  
  f();  
  digitalWrite(LED1, LOW);  
  f();  
}
```

```
void Thread1 (void) {  
  while (TRUE) {  
    f(); // operação longa  
  }  
}  
  
void Thread2 (void) {  
  while (TRUE) {  
    // pisca led!  
  }  
}
```

- sistema pode se tornar não reativo...
- quebrar tarefa longa em várias?
- usar threads?

Modelo Síncrono

- durante uma unidade de tempo lógico, o ambiente está invariante e não interrompe o programa
- Implementação:
 - **sampling**: arduino loop
 - **event-driven**: padrão observer

Hipótese síncrona

- “reações executam infinitamente mais rápido do que taxa de eventos”

padrão observador

```
on EVENT do  
  react()  
end
```

- ocorrência de evento executa “callback”

botão: button_changed()

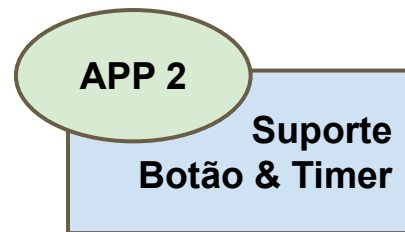
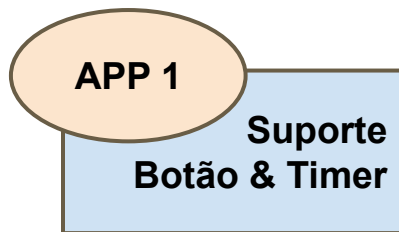
timer: timer_expired()

rede: data_received()

leitura de sensor: sensordata_ready()

padrão observador: exemplo

- programar no arduino um suporte para escrever programas orientados a eventos
- as funções **setup** e **loop** serão usadas para programar esse suporte
 - o arquivo com essas funções ficará *exatamente* igual para diferentes aplicações



exemplo observador: reescrevendo pisca/para

```
#include "pindefs.h"

void setup () {
    pinMode(LED1, OUTPUT);
    pinMode(KEY1, INPUT_PULLUP);
}

void loop() {
    digitalWrite(LED1, HIGH);
    delay(1000);
    digitalWrite(LED1, LOW);
    delay(1000);

    if (!digitalRead(KEY1)) {
        digitalWrite(LED1, LOW);
        exit(0);
    }
}
```

```
#include "eventdriven.h"
#include "app.h"
#include "pindefs.h"
int state = 0;
```

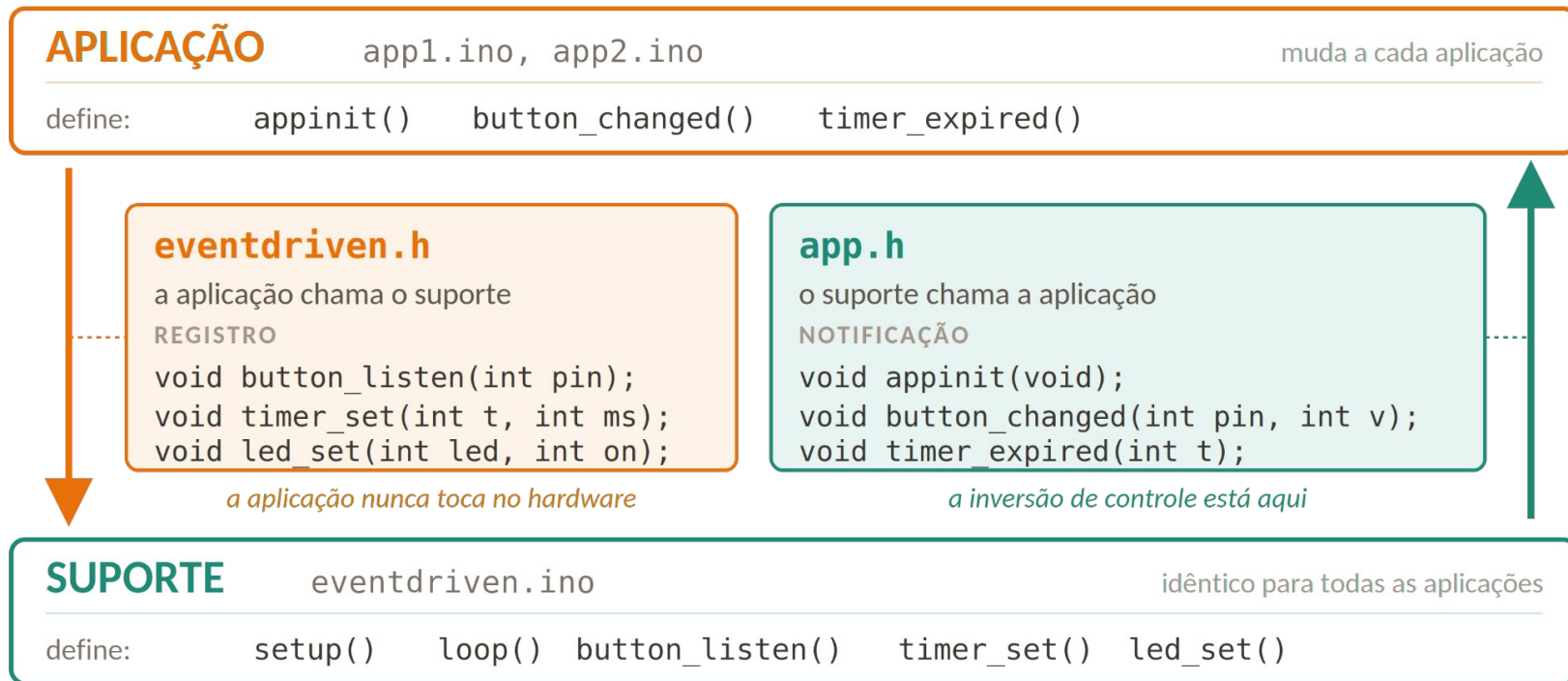
**Aplicação do usuário
do "suporte".**

```
void appinit(){
    button_listen(KEY1);
    timer_set (1, 1000);
}

void button_changed (int pin, int v) {
    if (v) { // pressionada
        led_set(LED1, 1);
        exit();
    }
}

void timer_expired (int t) {
    state = !state;
    led_set(LED1, state);
    timer_set (1, 1000);
}
```

padrão observador: dois arquivos, dois sentidos



exercício observador: arquivos de interface

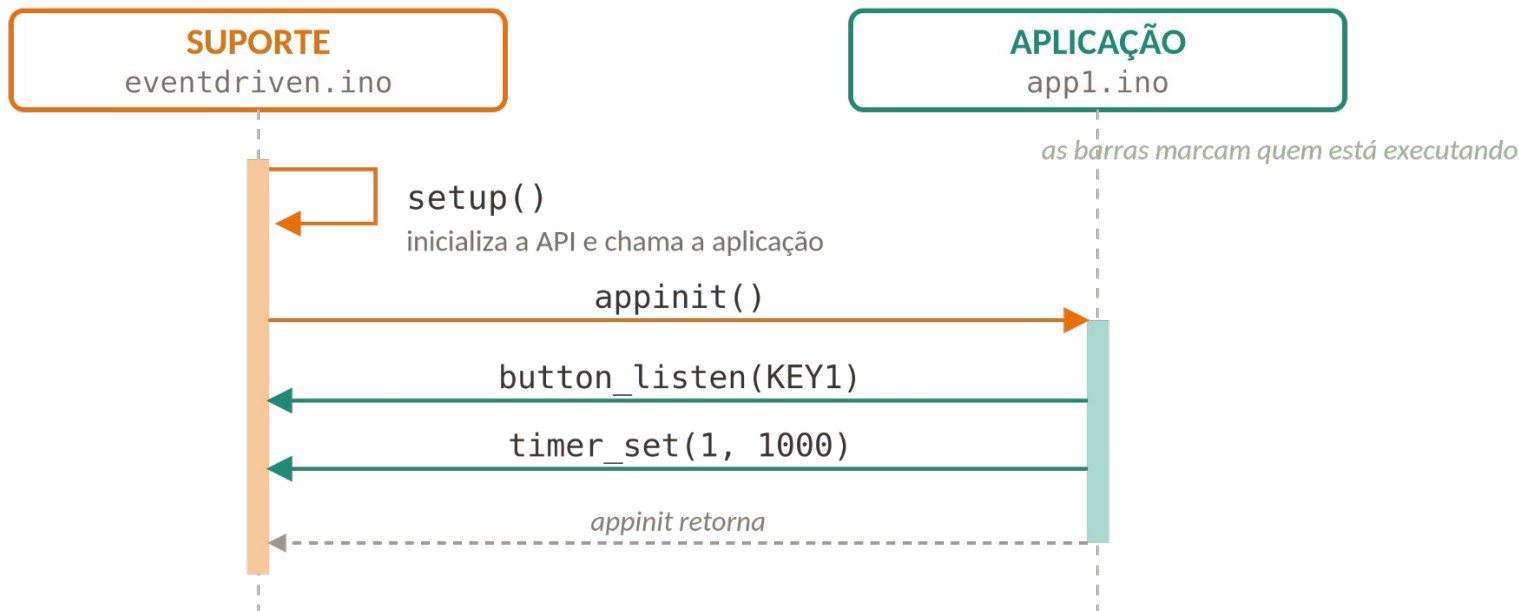
Eventdriven.h - a aplicação chama o suporte

```
/* aplicação quer ser notificada de alterações em 'pin' */  
void button_listen (int pin);  
  
/* liga timer 't' (0, 1 ou 2) para 'ms' milisegundos. timer só dispara uma vez */  
void timer_set (int t, int ms);  
  
/* acende (on = 1) ou apaga (on = 0) o led */  
void led_set (int led, int on);
```

App.h - o suporte chama a aplicação

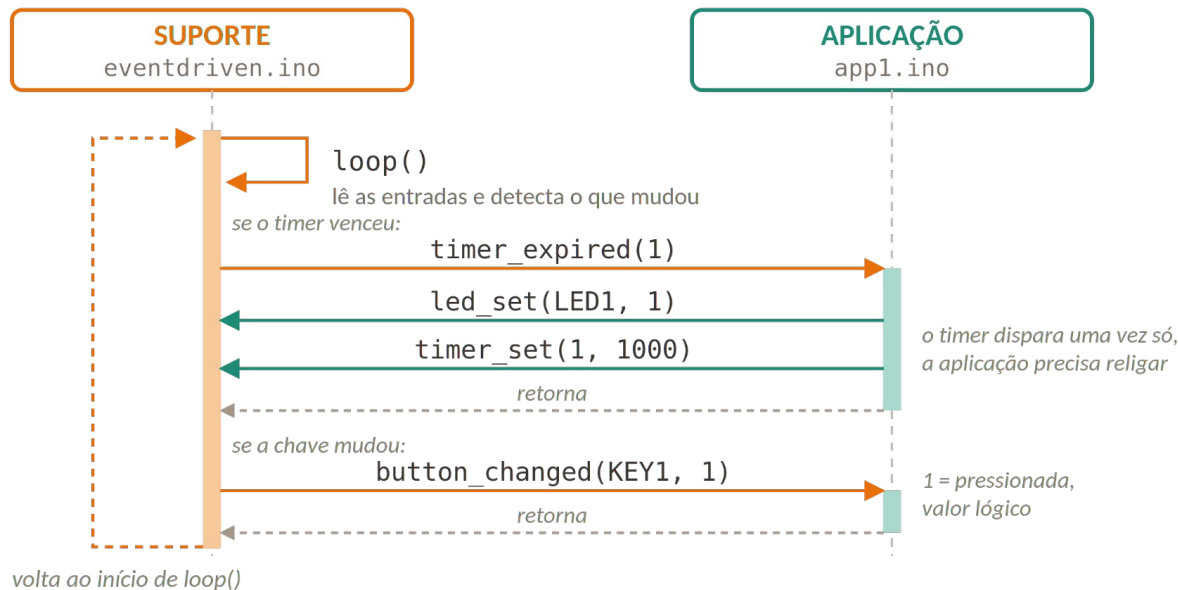
```
/* chama aplicação para iniciar configuração */  
void appinit(void);  
  
/* avisa que 'pin' passou a pressionado (v = 1) ou solto (v = 0) */  
void button_changed (int pin, int v);  
  
/* avisa que timer 't' expirou */  
void timer_expired(int t);
```

quem chama quem: inicialização



A aplicação declara o que quer observar e devolve o controle. Nada mais é chamado por ela.

quem chama quem: execução



- cada callback devolve o controle antes de loop continuar
- uma reação nunca interrompe a outra: é a hipótese síncrona

dois degraus de abstração

- `pindefs.h` já esconde qual pino. Ninguém escreve o número 13 no código.
- falta esconder o que o valor significa:
 - com `INPUT_PULLUP`, pressionada é `LOW`;
 - o LED acende em `LOW`
- hoje cada aplicação repete essa convenção, e basta uma inversão para o programa reagir à chave solta sem ninguém perceber
- com o suporte, a convenção elétrica existe em um único arquivo: `button_changed` entrega valor lógico, `led_set` recebe valor lógico

<code>if (!digitalRead(KEY1))</code> <code>digitalWrite(LED1, LOW);</code>	→	<code>if (v)</code> <code>led_set(LED1, 1);</code>
---	---	---

Se a placa mudar e o LED passar a acender em HIGH, quantos arquivos você altera?

padrão observador: esqueleto de implementação

```
/* Funções de registro: */  
void button_listen (int pin) {  
    <...>           // "pin" passado deve gerar notificações  
}  
void timer_set (int t, int ms) {  
    <...>           // timer t deve expirar após "ms" milisegundos  
}                   // timer só dispara uma vez  
void led_set (int led, int on) {  
    <...>           // acende (on = 1) ou apaga (on = 0) o led  
}  
  
/* Callbacks - definidas em app.h */  
void button_changed (int pin, int v); // notifica que "pin" mudou para "v"  
void timer_expired (int t);          // notifica que o timer expirou  
  
/* Programa principal: */  
void setup () {  
    <...>           // inicialização da API  
    appinit();      // inicialização do usuário  
}
```

```
void loop () {  
    <...>           // detecta novos eventos  
    button_changed(...); // notifica o usuário  
    <...>           // detecta novos eventos  
    timer_expired(...);  // notifica o usuário  
}
```

Tarefa Concorrência (1 de 2)

1. Crie as interfaces `eventdriven.h` e `app.h` como as indicadas.
2. Crie `eventdriven.ino` implementando `button_listen`, `timer_set`, `led_set`, `setup` e `loop`. `setup` deve chamar `appinit`. `loop` deve detectar as mudanças registradas e notificá-las pelas callbacks. Atenção a três pontos:
 - `digitalRead` devolve o nível do pino, não a transição: guarde o último valor lido de cada pino registrado
 - o ruído de contato, visto na aula anterior, é tratado pelo suporte, com marca de tempo por pino e sem bloquear o loop
 - o suporte é o único lugar que conhece as convenções elétricas da placa; ele entrega e recebe valores lógicos
3. Escreva `app1.ino` com o código mostrado à direita do slide 26, como primeiro teste.

Tarefa Concorrência (2 de 2)

4. Sem alterar os arquivos `.h` nem `eventdriven.ino`, reescreva a Tarefa 1 da aula de Arduino como `app2.ino`: pisca continuamente, a chave 1 divide o período por 2, a chave 2 dobra o período, com limites inferior e superior, e o pisca para ou volta a piscar quando as chaves 1 e 2 forem pressionadas com intervalo menor que 500 ms.
 - a aplicação não pode tocar no hardware: nada de `millis()`, `digitalRead` ou `digitalWrite`
 - existem três temporizadores, e o pisca usa apenas um deles
5. Compare `app2.ino` com a sua solução da aula anterior. Onde ficou o controle de tempo em cada uma? Se o LED passasse a acender em HIGH, quantos arquivos você alteraria em cada solução?

Cada aplicação é um sketch separado, em sua própria pasta: `app1/app1.ino` e `app2/app2.ino`. A IDE junta todos os `.ino` de uma pasta antes de compilar, então as duas aplicações juntas dariam funções duplicadas. Copie `eventdriven.ino`, `eventdriven.h` e `app.h` para as duas pastas, sem alterar nada. Os dois `eventdriven.ino` devem ficar idênticos.